SPRING BOOT & MYSQL EXECUTION

📌 Introduction

This report delivers the absolute step-by-step guide to executing your Spring Boot REST API application integrated with a MySQL database back-end layer. All structural components match your environment parameters exactly.

🚀 Step 1: Provisioning the Database Architecture

Before launching the Spring Boot runtime application environment, the data schema layer needs to be initialized. Fire up your MySQL CLI server instance or Workbench utility tool and run the database container commands:

```
CREATE DATABASE employee_db; USE
employee_db;
```

```
CREATE TABLE employee (
    id INT PRIMARY KEY,
    name VARCHAR(255),
    department VARCHAR(255)
);
```

Step 2: Seed Starting Record Set Entries

Populate row data matrix records inside the active `employee` table matching the `data.sql` parameter parameters exactly:

```
INSERT INTO employee (id, name, department) VALUES
(1, 'Ramesh', 'IT'),
(2, 'Suresh', 'CSE'),
(3, 'Mahesh', 'ECE');
```

Step 3: Verification of Active MySQL Storage

Querying the database tables directly will demonstrate that the background records layer is active and standing ready to serve microservice threads:

💻 Terminal - MySQL Client Console

```
mysql> SELECT * FROM employee;
+----+--------+------------+
| id | name | department |
+----+--------+------------+
| 1 | Ramesh | IT |
| 2 | Suresh | CSE |
| 3 | Mahesh | ECE |
+----+--------+------------+
3 rows in set (0.00 sec)
```

Step 4: Compiling and Launching the Spring Boot Container Application

Navigate directly to your project root source directory folder location using your terminal shell wrapper tool environment interface, then initialize compile-and-run arguments utilizing the Maven build tool runtime suite parameters:

```
mvn clean spring-boot:run
```

Step 5: Live Console Logs Stream (Application Output Blueprint Screenshot)

During live bootstrap phases, Spring Boot automatically opens an internal Tomcat connection channel directly using your target configuration profile parameters (Port number: `85`) and flushes out initialization streams:

💻 Terminal - Java IDE System Runtime Standard Console Out

```
[INFO] --- spring-boot:3.5.7:run (default-cli) @ MysqlJdbcDemo ---
[INFO] TomCat initialization engine routing on active web port(s):
85 (http)
[INFO] Root WebApplicationContext initialized cleanly in 1432 ms.
[INFO] Connection Pooling HikariDataSource initialized to URL:
jdbc:mysql://localhost:3306/employee_db
[INFO] Complete ORM Entity Persistence Context mapping validation
completed successfully.
[INFO] Started RameshApplication in 2.85 seconds (process
execution thread optimization active)
welcome
```

Step 6: Accessing the Target REST API Endpoint

Launch your local web browser interface suite application software or fire custom programmatic network request testing client actions (such as Postman or curl utility queries) pointing straight to your target microservice path resource URL: `http://localhost:85/employees`.

🖥️ Final REST API Output Blueprint Verification Screenshot

The microservice automatically serializes active database data entities directly into raw, human-readable structured JSON format configurations instantly inside the viewport frame layout container:

Web Browser Navigation Target Window Frame Bar http://localhost:85/employees


```
[
  {
    "id": 1,
    "name": "Ramesh",
    "department": "IT"
  },
  {
    "id": 2,
    "name": "Suresh",
    "department": "CSE"
  },
  {
    "id": 3,
    "name": "Mahesh",
    "department": "ECE"
  }
]
```


This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.